What is **idempotency** ? need full example web api program

Worked for 17s

What is Idempotency?

Idempotency means sending the same request multiple times produces the same final result as sending it once.

For example, a customer clicks **Place Order**, but the response is delayed. The client retries the request:

```
POST /api/orders  → Order created: 1001
POST /api/orders  → Returns existing Order: 1001
POST /api/orders  → Returns existing Order: 1001
```

Only **one order** is created because every retry contains the same `Idempotency-Key` .

HTTP Methods

Method	Idempotent?	Reason
GET	Yes	Reading repeatedly does not change data
PUT	Yes	Replaces the resource with the same state

Method	Idempotent?	Reason
DELETE	Yes	Resource remains deleted
PATCH	Depends	Some patch operations may repeatedly change data
POST	No, by default	Every request may create a new resource

POST can be made idempotent by using an **Idempotency-Key**.

Complete ASP.NET Core Web API Example

This example uses:

- ASP.NET Core Web API
- Controllers
- Entity Framework Core
- SQL Server
- `Idempotency-Key` request header
- Database transaction
- Unique constraint to prevent concurrent duplicates

1. Create the project

Bash

Install Entity Framework Core packages:

Bash

2. Project structure

```
IdempotencyDemo
│
├── Controllers
│   └── OrdersController.cs
├── Data
│   └── AppDbContext.cs
├── DTOs
│   ├── CreateOrderRequest.cs
│   └── OrderResponse.cs
├── Models
│   ├── Order.cs
│   └── IdempotencyRecord.cs
├── Program.cs
└── appsettings.json
```

3. Order model

Models/Order.cs

```
namespace IdempotencyDemo.Models;

public class Order
{
    public int Id { get; set; }
```

```
    public string CustomerName { get; set; } = string.Empty;

    public string ProductName { get; set; } = string.Empty;

    public int Quantity { get; set; }

    public decimal TotalAmount { get; set; }
```

C#

4. Idempotency record model

This table remembers which requests have already been processed.

Models/IdempotencyRecord.cs

```
namespace IdempotencyDemo.Models;

public class IdempotencyRecord
{
    public int Id { get; set; }

    public string IdempotencyKey { get; set; } = string.Empty;

    public string ResponseBody { get; set; } = string.Empty;

    public int StatusCode { get; set; }
```

C#

5. Request DTO

DTOs/CreateOrderRequest.cs

```C#
using System.ComponentModel.DataAnnotations;

namespace IdempotencyDemo.DTOs;

public class CreateOrderRequest
{
    [Required]
    public string CustomerName { get; set; } = string.Empty;

    [Required]
    public string ProductName { get; set; } = string.Empty;

    [Range(1, 100)]
    public int Quantity { get; set; }

    [Range(0.01, double.MaxValue)]
```

6. Response DTO

DTOs/OrderResponse.cs

```C#
namespace IdempotencyDemo.DTOs;

public class OrderResponse
{
```

```
    public int OrderId { get; set; }

    public string CustomerName { get; set; } = string.Empty;

    public string ProductName { get; set; } = string.Empty;

    public int Quantity { get; set; }

    public decimal TotalAmount { get; set; }

    public DateTime CreatedAt { get; set; }
```

C#

7. Database context

Data/AppDbContext.cs

```
using IdempotencyDemo.Models;
using Microsoft.EntityFrameworkCore;

namespace IdempotencyDemo.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Order> Orders => Set<Order>();
```

```
    public DbSet<IdempotencyRecord> IdempotencyRecords =>
        Set<IdempotencyRecord>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<IdempotencyRecord>()
            .HasIndex(x => x.IdempotencyKey)
            .IsUnique();
```

C#

The unique index ensures that two simultaneous requests cannot save the same key.

8. Orders controller

Controllers/OrdersController.cs

```
using System.Text.Json;
using IdempotencyDemo.Data;
using IdempotencyDemo.DTOs;
using IdempotencyDemo.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace IdempotencyDemo.Controllers;

[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly AppDbContext _context;

    public OrdersController(AppDbContext context)
```

```
    {
        _context = context;
    }

    [HttpPost]
    public async Task<ActionResult<OrderResponse>> CreateOrder(
        [FromBody] CreateOrderRequest request,
        CancellationToken cancellationToken)
    {
        string? idempotencyKey =
            Request.Headers["Idempotency-Key"].FirstOrDefault();

        if (string.IsNullOrWhiteSpace(idempotencyKey))
        {
            return BadRequest(new
            {
                message = "Idempotency-Key header is required."
            });
        }

        // Check whether this request was already processed.
        var existingRecord = await _context.IdempotencyRecords
            .AsNoTracking()
            .FirstOrDefaultAsync(
                x => x.IdempotencyKey == idempotencyKey,
                cancellationToken);

        if (existingRecord is not null)
        {
            var previousResponse =
                JsonSerializer.Deserialize<OrderResponse>(
                    existingRecord.ResponseBody);

            return StatusCode(
                existingRecord.StatusCode,
```

```
            previousResponse);
    }

    await using var transaction =
        await _context.Database.BeginTransactionAsync(
            cancellationToken);

    try
    {
        var order = new Order
        {
            CustomerName = request.CustomerName,
            ProductName = request.ProductName,
            Quantity = request.Quantity,
            TotalAmount = request.TotalAmount,
            CreatedAt = DateTime.UtcNow
        };

        _context.Orders.Add(order);

        await _context.SaveChangesAsync(cancellationToken);

        var response = new OrderResponse
        {
            OrderId = order.Id,
            CustomerName = order.CustomerName,
            ProductName = order.ProductName,
            Quantity = order.Quantity,
            TotalAmount = order.TotalAmount,
            CreatedAt = order.CreatedAt,
            Message = "Order created successfully."
        };

        var idempotencyRecord = new IdempotencyRecord
        {
```

```
                IdempotencyKey = idempotencyKey,
                ResponseBody = JsonSerializer.Serialize(response),
                StatusCode = StatusCodes.Status201Created,
                CreatedAt = DateTime.UtcNow
            };

            _context.IdempotencyRecords.Add(idempotencyRecord);

            await _context.SaveChangesAsync(cancellationToken);
            await transaction.CommitAsync(cancellationToken);

            return CreatedAtAction(
                nameof(GetOrder),
                new { id = order.Id },
                response);
        }
        catch (DbUpdateException)
        {
            await transaction.RollbackAsync(cancellationToken);

            // Another concurrent request may have used the same key.
            _context.ChangeTracker.Clear();

            var completedRecord = await _context.IdempotencyRecords
                .AsNoTracking()
                .FirstOrDefaultAsync(
                    x => x.IdempotencyKey == idempotencyKey,
                    cancellationToken);

            if (completedRecord is not null)
            {
                var previousResponse =
                    JsonSerializer.Deserialize<OrderResponse>(
                        completedRecord.ResponseBody);
```

```
                return StatusCode(
                    completedRecord.StatusCode,
                    previousResponse);
            }

            throw;
        }
    }

    [HttpGet("{id:int}")]
    public async Task<ActionResult<Order>> GetOrder(
        int id,
        CancellationToken cancellationToken)
    {
        var order = await _context.Orders
            .AsNoTracking()
            .FirstOrDefaultAsync(
                x => x.Id == id,
                cancellationToken);

        return order is null ? NotFound() : Ok(order);
    }

    [HttpGet]
    public async Task<ActionResult<List<Order>>> GetOrders(
        CancellationToken cancellationToken)
    {
        var orders = await _context.Orders
            .AsNoTracking()
            .OrderByDescending(x => x.CreatedAt)
```

‹/› C#

```
        return Ok(orders);
    }
}
```

9. Program.cs

```
using IdempotencyDemo.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString(
            "DefaultConnection")));

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.MapControllers();
```

‹/› C#

10. Connection string

appsettings.json

```json
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=IdempotencyDemoDb;Trusted_
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
```

JSON

11. Create the database

```bash
dotnet ef migrations add InitialCreate
```

Bash

Testing the API

First request

```http
Idempotency-Key: order-request-1001
```

```json
{
  "customerName": "Syed",
  "productName": "Laptop",
  "quantity": 1,
```

Response:

```json
{
  "orderId": 1,
  "customerName": "Syed",
  "productName": "Laptop",
  "quantity": 1,
  "totalAmount": 65000,
  "createdAt": "2026-09-14T10:30:00Z",
```

Status:

```
201 Created
```

Retry with the same key

Send the exact request again:

```http
```

The API returns the stored response containing:

```json
{
```

It does not create Order 2.

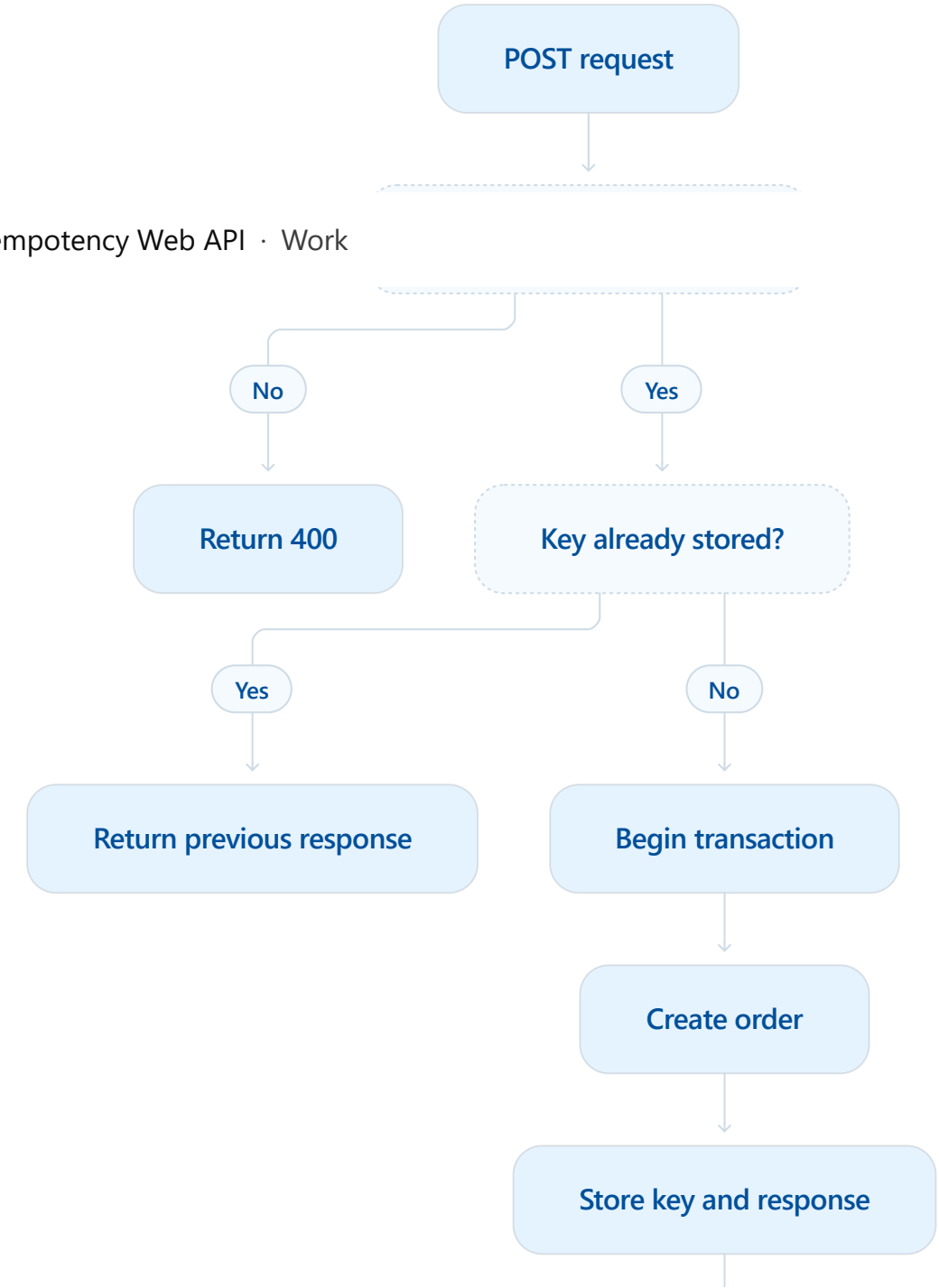
New request

Change the key:

```http
```

Now the API creates a new order.

Request Flow



Explain Idempotency Web API · Work

Share

Outputs

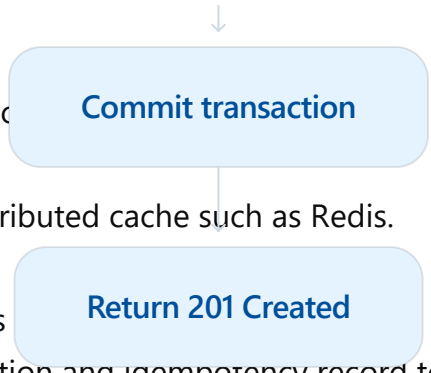
Create file or site

Sources

Memory

Important Production Points

- The client must generate a unique key for eac
- The same retry must use the same key.
- Store the key in a persistent database or distributed cache such as Redis.
- Create a unique database index on the key.
- Store the status code and response so retries
- Use a transaction to save the business operation and idempotency record together.
- Add an expiration time and periodically remove old records.
- For payment APIs, also pass the idempotency key to the payment provider.
- Ideally, store a request hash and reject the same key if it is reused with different request data.





ChatGPT can make mistakes. Check important info.